

FCP – Unit 3 Notes (Functions, Recursion, Storage Classes, Pointers)

1. Functions in C

A **function** is a block of code designed to perform a specific task. Functions help divide a large program into smaller and manageable parts.

Advantages of Functions

- Code reusability
- Reduces program length
- Improves readability
- Easier debugging and testing

Types of Functions

1. **Library Functions**
2. **User Defined Functions**

Example of library functions:

- `printf()`
 - `scanf()`
 - `strlen()`
 - `sqrt()`
-

2. Function Definition

Function definition specifies what the function does.

Syntax

```
return_type function_name(parameters)
{
    // statements
    return value;
}
```

Example:

```
#include<stdio.h>
```

```
int add(int a, int b)
{
    int sum;
    sum = a + b;
    return sum;
}
```

```
int main()
{
    int result;
    result = add(5,3);
    printf("Sum = %d", result);
}
```

Output

Sum = 8

3. Function Declaration (Prototype)

Function declaration tells the compiler about the function before it is used.

Example:

```
int add(int, int);
```

This informs the compiler that:

- Function name = `add`
 - It takes two integers
 - It returns an integer
-

4. Function Call

A **function call** invokes the function to perform its task.

Example

```
result = add(5,3);
```

Steps when function is called:

1. Control transfers to function
 2. Function executes
 3. Value returns to calling function
-

5. Types of Functions Based on Arguments and Return Value

1. No Arguments, No Return Value

```
void display()  
{  
    printf("Hello");  
}
```

2. Arguments, No Return Value

```
void add(int a, int b)  
{  
    printf("%d", a+b);  
}
```

```
}
```

3. Arguments with Return Value

```
int add(int a, int b)
{
    return a+b;
}
```

4. No Arguments with Return Value

```
int number()
{
    return 5;
}
```

6. Recursion

Recursion occurs when a function calls itself.

It is used to solve problems that can be broken into smaller similar problems.

Example: Factorial

Factorial Formula

$$n! = n \times (n-1)!$$

Example Program

```
#include<stdio.h>
```

```
int factorial(int n)
{
    if(n==0)
        return 1;
    else
        return n * factorial(n-1);
}
```

```
int main()
{
```

```
int num = 5;
printf("Factorial = %d", factorial(num));
}
```

Output

Factorial = 120

Advantages of Recursion

- Shorter code
- Easier to solve mathematical problems

Disadvantages

- Uses more memory
 - Slower than loops
-

7. Storage Classes in C

Storage classes define:

- Scope of variable
- Lifetime
- Visibility

Types of Storage Classes

1. `auto`
2. `register`
3. `static`

4. `extern`

1. Auto Storage Class

- Default storage class
- Scope: inside function
- Lifetime: until function ends

Example

```
void func()
{
    auto int x = 10;
}
```

2. Register Storage Class

- Stores variable in CPU register
- Faster access

Example

```
register int x;
```

Limitations

- Cannot use `&` operator
-

3. Static Storage Class

- Variable retains value between function calls.

Example

```
#include<stdio.h>
```

```
void count()
{
    static int x = 0;
    x++;
    printf("%d\n", x);
}
```

```
int main()
{
    count();
    count();
    count();
}
```

Output

```
1
2
3
```

4. Extern Storage Class

- Used to access global variable across files.

Example

```
extern int x;
```

8. Pointers in C

A **pointer** is a variable that stores the memory address of another variable.

Example

```
int a = 10;  
int *p;
```

```
p = &a;
```

Here:

- **a** stores value
- **p** stores address of **a**

Pointer Operators

Address Operator **&**

Returns memory address.

&a

Dereference Operator *****

Accesses value stored at address.

***p**

Example

```
#include<stdio.h>
```

```
int main()  
{  
    int a = 10;  
    int *p;  
  
    p = &a;  
  
    printf("Value of a = %d\n", a);  
    printf("Address of a = %p\n", &a);  
    printf("Value using pointer = %d", *p);  
}
```

9. Pointer Arithmetic

Pointers can perform arithmetic operations.

Example

```
p++  
p--  
p+1  
p-1
```

If pointer points to integer:

- Increment moves by **4 bytes**
-

10. Pointers and Arrays

Array name itself acts as a pointer.

Example

```
int arr[3] = {10,20,30};  
  
printf("%d", *(arr+1));
```

Output

20

11. Passing Pointers to Functions

Pointers allow **call by reference**.

Example

```
#include<stdio.h>
```

```
void swap(int *a, int *b)
```

```

{
    int temp;
    temp = *a;
    *a = *b;
    *b = temp;
}

int main()
{
    int x=10, y=20;

    swap(&x,&y);

    printf("%d %d", x,y);
}

```

Output

20 10

12. Function Pointers

A pointer can store address of a function.

Example

```

int add(int a,int b)
{
    return a+b;
}

```

```

int (*p)(int,int);

```

```

p = add;

```

Summary

Unit 3 Topics:

- Functions in C
- Function declaration and calling
- Recursion
- Storage classes
- Pointers
- Pointer arithmetic
- Pointers with arrays
- Passing pointers to functions
- Function pointers

FCP – Unit 3 Notes (Functions, Recursion, Storage Classes, Pointers)

1. Functions in C

A **function** is a block of code designed to perform a specific task. Functions help divide a large program into smaller and manageable parts.

Advantages of Functions

- Code reusability
- Reduces program length
- Improves readability
- Easier debugging and testing

Types of Functions

1. **Library Functions**
2. **User Defined Functions**

Example of library functions:

- `printf()`
 - `scanf()`
 - `strlen()`
 - `sqrt()`
-

2. Function Definition

Function definition specifies what the function does.

Syntax

```
return_type function_name(parameters)
{
    // statements
    return value;
}
```

Example:

```
#include<stdio.h>
```

```
int add(int a, int b)
{
    int sum;
    sum = a + b;
    return sum;
}
```

```
int main()
{
    int result;
```

```
    result = add(5,3);  
    printf("Sum = %d", result);  
}
```

Output

Sum = 8

3. Function Declaration (Prototype)

Function declaration tells the compiler about the function before it is used.

Example:

```
int add(int, int);
```

This informs the compiler that:

- Function name = **add**
 - It takes two integers
 - It returns an integer
-

4. Function Call

A **function call** invokes the function to perform its task.

Example

```
result = add(5,3);
```

Steps when function is called:

1. Control transfers to function
2. Function executes
3. Value returns to calling function

5. Types of Functions Based on Arguments and Return Value

1. No Arguments, No Return Value

```
void display()  
{  
    printf("Hello");  
}
```

2. Arguments, No Return Value

```
void add(int a, int b)  
{  
    printf("%d", a+b);  
}
```

3. Arguments with Return Value

```
int add(int a, int b)  
{  
    return a+b;  
}
```

4. No Arguments with Return Value

```
int number()  
{  
    return 5;  
}
```

6. Recursion

Recursion occurs when a function calls itself.

It is used to solve problems that can be broken into smaller similar problems.

Example: Factorial

Factorial Formula

$$n! = n \times (n-1)!$$

Example Program

```
#include<stdio.h>
```

```
int factorial(int n)
{
    if(n==0)
        return 1;
    else
        return n * factorial(n-1);
}
```

```
int main()
{
    int num = 5;
    printf("Factorial = %d", factorial(num));
}
```

Output

Factorial = 120

Advantages of Recursion

- Shorter code
- Easier to solve mathematical problems

Disadvantages

- Uses more memory
- Slower than loops

7. Storage Classes in C

Storage classes define:

- Scope of variable
- Lifetime
- Visibility

Types of Storage Classes

1. `auto`
 2. `register`
 3. `static`
 4. `extern`
-

1. Auto Storage Class

- Default storage class
- Scope: inside function
- Lifetime: until function ends

Example

```
void func()
{
    auto int x = 10;
}
```

2. Register Storage Class

- Stores variable in CPU register

- Faster access

Example

```
register int x;
```

Limitations

- Cannot use `&` operator
-

3. Static Storage Class

- Variable retains value between function calls.

Example

```
#include<stdio.h>
```

```
void count()
{
    static int x = 0;
    x++;
    printf("%d\n", x);
}
```

```
int main()
{
    count();
    count();
    count();
}
```

Output

```
1
2
3
```

4. Extern Storage Class

- Used to access global variable across files.

Example

```
extern int x;
```

8. Pointers in C

A **pointer** is a variable that stores the memory address of another variable.

Example

```
int a = 10;  
int *p;
```

```
p = &a;
```

Here:

- **a** stores value
 - **p** stores address of **a**
-

Pointer Operators

Address Operator **&**

Returns memory address.

```
&a
```

Dereference Operator *****

Accesses value stored at address.

*p

Example

```
#include<stdio.h>
```

```
int main()
{
    int a = 10;
    int *p;

    p = &a;

    printf("Value of a = %d\n", a);
    printf("Address of a = %p\n", &a);
    printf("Value using pointer = %d", *p);
}
```

9. Pointer Arithmetic

Pointers can perform arithmetic operations.

Example

```
p++
p--
p+1
p-1
```

If pointer points to integer:

- Increment moves by **4 bytes**
-

10. Pointers and Arrays

Array name itself acts as a pointer.

Example

```
int arr[3] = {10,20,30};
```

```
printf("%d", *(arr+1));
```

Output

20

11. Passing Pointers to Functions

Pointers allow **call by reference**.

Example

```
#include<stdio.h>
```

```
void swap(int *a, int *b)
{
    int temp;
    temp = *a;
    *a = *b;
    *b = temp;
}
```

```
int main()
{
    int x=10, y=20;

    swap(&x,&y);

    printf("%d %d", x,y);
}
```

Output

20 10

12. Function Pointers

A pointer can store address of a function.

Example

```
int add(int a,int b)
{
    return a+b;
}
```

```
int (*p)(int,int);
```

```
p = add;
```

Summary

Unit 3 Topics:

- Functions in C
- Function declaration and calling
- Recursion
- Storage classes
- Pointers
- Pointer arithmetic
- Pointers with arrays
- Passing pointers to functions
- Function pointers